FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Eléctrica y Electrónica
	Carrera: Electrónica y Automatización Asignatura: Fundamentos de la Programación NRC:28561
	Apellidos y nombres del estudiante: Figueroa Cerón Sofía Vanessa

Ejercicio 16

Ejercicio:

16. Triángulo Rectángulo Hueco

Resultado esperado	Imprimir un triángulo rectángulo hueco de 10 líneas usando asteriscos (*), dejando vacío el interior.

Contexto del ejercicio

En el documento base se presenta un programa en C que solicita el número de líneas y dibuja un triángulo hueco. La condición clave es imprimir asterisco cuando la posición esté en el borde izquierdo, en la diagonal o en la última fila; caso contrario se imprime espacio.

Regla de impresión: si j = 1, j = i o i = n, se imprime "*"; en cualquier otro caso se imprime un espacio.

1. Seudocódigo para PSeInt

```
Proceso Triangulo

   Definir n, i, j Como Entero

   Repetir
      Escribir "Lineas del triangulo: "
      Leer n

      Si n <= 0 Entonces
         Escribir "Ingrese un numero mayor que 0"
      FinSi

   Hasta Que n > 0
```

```
  Para i <- 1 Hasta n Con Paso 1 Hacer

    Para j <- 1 Hasta i Con Paso 1 Hacer

      Si j = 1 O j = i O i = n Entonces
        Escribir Sin Saltar "* "
      SiNo
        Escribir Sin Saltar "  "
      FinSi

    FinPara

    Escribir ""

  FinPara

FinProceso
```

2. Prueba de escritorio en PSeInt

Fila i	Rango de j	Condición que dibuja borde	Salida de la fila	Explicación breve
1	1	j=1 y j=i	*	Primera fila: solo un asterisco.
2	1 a 2	j=1 o j=i	* *	Tiene borde izquierdo y diagonal.
3	1 a 3	j=1 o j=i	* *	El centro queda vacío.
4	1 a 4	j=1 o j=i	* *	Aumenta el espacio interno.
5	1 a 5	j=1 o j=i	* *	Se mantienen los bordes.
6	1 a 6	j=1 o j=i	* *	Continúa el patrón hueco.
7	1 a 7	j=1 o j=i	* *	Más columnas internas vacías.
8	1 a 8	j=1 o j=i	* *	El borde diagonal se desplaza.

9	1 a 9	j=1 o j=i	* *	Última fila antes de la base.
10	1 a 10	i=n	* * * * * * * * * *	Base completa del triángulo.

Salida esperada al ingresas 10:
Lineas del triangulo:
10

```
*
* *
*  *
*   *
*    *
*     *
*      *
*       *
*        *
* * * * * * * * * *
```

Fila i	Rango de j	Condición que dibuja	Salida	Descripción
1	1	j=1, j=i	*	Primera fila: solo un asterisco.
2	1 a2	j=1 o j=i	**	Tiene borde izquierdo y diagonal.
3	1 a 3	j=1 o j=i	**	El centro queda vacío.
4	1 a 4	j=1 o j=i	**	Aumenta el espacio interno.
5	1 a 5	i=n	*****	Base completa del triángulo.

Salida esperada al ingresar:

Lineas del triangulo:

5

```
*
* *
```

```
*  *

*   *

* * * * *
```

Fila i	Rango en j	Condición que dibuja	Salida	Descripción
1	1	j=1, j=i	*	Primera fila: solo un asterisco.
2	1 a2	j=1 o j=i	* *	Tiene borde izquierdo y diagonal.
3	1 a 3	j=1 o j=i	* *	El centro queda vacío.
4	1 a 4	j=1 o j=i	* *	Aumenta el espacio interno.
5	1 a 5	j=1 o j=i	* *	Sigue formando el triángulo hueco.
6	1 a 6	j=1 o j=i	* *	Se mantiene el borde izquierdo y diagonal.
7	1 a 7	i=n	* * * * * * *	Base completa del triángulo.

Salida esperada al ingresar:

Lineas del triangulo:

7

```
*

* *

*  *

*   *

*    *

*     *

* * * * * *
```

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

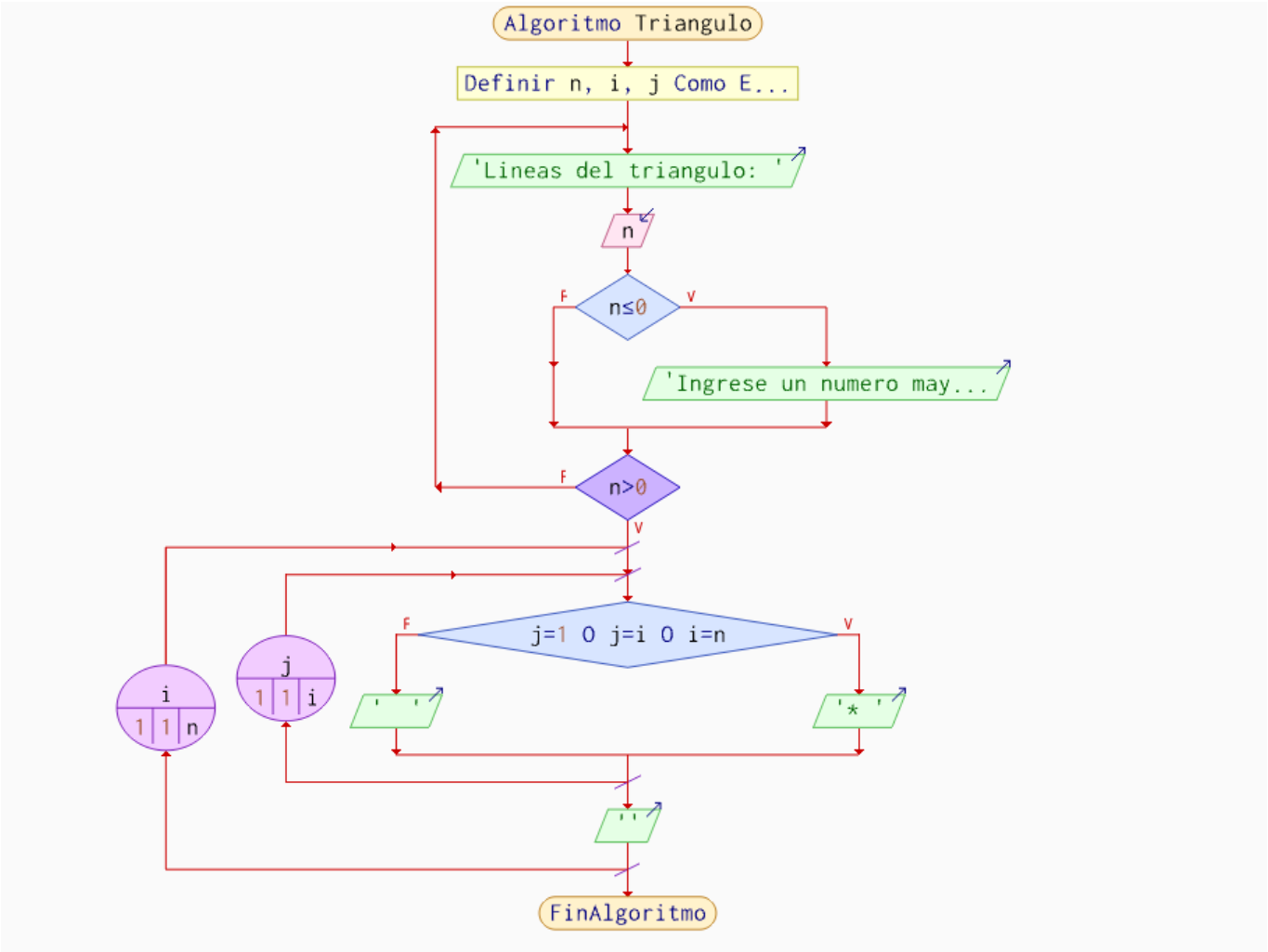
PSeInt	C / Code::Blocks	Para recordar
Proceso Triangulo	int main() {	Todo programa en C inicia con main.
Definir n, i, j Como Entero	int n, i, j;	Las variables enteras en C se declaran con int.
Repetir	for (;;) {	Se usa para repetir sin usar Mientras ni do while.
Escribir "Lineas del triangulo:"	printf("Lineas del triangulo: ");	printf muestra mensajes en pantalla.
Leer n	scanf("%d", &n);	%d lee números enteros.
Si n <= 0 Entonces	if (n <= 0) {	Se valida que el número sea mayor que cero.
Escribir "Ingrese un numero mayor que 0"	printf("Ingrese un numero mayor que 0\n");	Mensaje cuando el dato no es válido.
Hasta Que n > 0	if (n > 0) { break; }	El ciclo termina cuando n es mayor que cero.
Para i <- 1 Hasta n Con Paso 1 Hacer	for (i = 1; i <= n; i++) {	i representa la fila del triángulo.
Para j <- 1 Hasta i Con Paso 1 Hacer	for (j = 1; j <= i; j++) {	j representa la columna dentro de cada fila.
Si j = 1 O j = i O i = n Entonces	`if (j == 1	
Escribir Sin Saltar "* "	printf("* ");	Imprime asteriscos sin cambiar de línea.
SiNo	else {	Se ejecuta cuando no se cumple la condición.
Escribir Sin Saltar " "	printf(" ");	Imprime espacios para dejar el triángulo hueco.
Escribir ""	printf("\n");	Hace el salto de línea después de cada fila.
FinSi / FinPara / FinProceso	}	Las llaves cierran condiciones, ciclos y el programa.

Código validado en C para Code::Blocks

```
#include <stdio.h>


int main() {
    int n, i, j;

    for (;;) {
        printf("Lineas del triangulo: ");
        scanf("%d", &n);

        if (n <= 0) {
            printf("Ingrese un numero mayor que 0\n");
        }

        if (n > 0) {
            break;
        }
    }

    for (i = 1; i <= n; i++) {

        for (j = 1; j <= i; j++) {

            if (j == 1 || j == i || i == n) {
                printf("* ");
            } else {
                printf("  ");
            }
```

```
        }

        printf("\n");
    }

    return 0;
}
```

Diagrama De Flujo



Conclusión

El programa permite dibujar correctamente un triángulo rectángulo hueco utilizando asteriscos. Primero valida que el número de líneas sea mayor que cero y luego usa ciclos Para para recorrer las filas y columnas. La condición j = 1 O j = i O i = n permite imprimir los bordes del triángulo, mientras que en las posiciones internas se imprimen espacios.

Recomendación

Se recomienda validar siempre que el valor ingresado sea positivo antes de dibujar el triángulo. También es importante recordar que i representa las filas y j representa las columnas, ya que eso ayuda a entender mejor cómo se forma la figura. En C, se debe usar || para representar el operador lógico **O**.

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c o abrir el archivo del ejercicio.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.F9
5. Comparar la salida con la prueba de escritorio.

4. Rúbrica de evaluación sobre 20 puntos

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Identifica claramente que el triángulo debe ser hueco, con borde izquierdo, diagonal y base.	Reconoce la forma del triángulo, aunque explica parcialmente la condición de impresión.	Confunde triángulo lleno con triángulo hueco o no distingue los bordes.	4 pts
Seudocódigo en PSeInt	Usa correctamente variables, Repetir, Para, Si/Sino y Escribir Sin Saltar.	Presenta la estructura general, con pequeños errores de sintaxis o ubicación.	El seudocódigo está incompleto o no representa la lógica del ejercicio.	4 pts
Prueba de escritorio	Comprueba el algoritmo fila por fila y explica la salida para n = 10.	Realiza una prueba parcial con algunas filas o poca explicación.	No evidencia seguimiento de variables i y j o la salida no coincide.	4 pts
Traducción a C en Code::Blocks	El código compila, ejecuta y mantiene la lógica validada del triángulo hueco.	El código tiene errores menores corregibles, pero la lógica principal es adecuada.	El código no compila o no aplica correctamente FOR, if y operadores lógicos.	5 pts
Presentación y orden	Entrega el trabajo limpio, ordenado y fácil de leer, con comentarios útiles.	El trabajo es comprensible, aunque puede mejorar en orden o comentarios.	El trabajo es difícil de seguir o no evidencia organización.	3 pts